AMERICAN INTERNATIONAL UNIVERSITY – BANGLADESH(AIUB)

Where leaders are created

Power Management and Sleep Mode Features of ATMEGA328P

Power Management and Sleep Modes

Power consumption is a critical issue for a device running continuously for a long time without being turned off. So, to overcome this problem almost every controller comes with a **Sleep Mode**, which helps developers to design electronic gadgets. Sleep mode puts the device in power-saving mode optimal **power consumption** by turning off the unused modules.

An Arduino Sleep mode is also referred to as **Arduino Power Save Mode** or **Arduino Standby Mode**.

Arduino UNO, Arduino Nano, and Pro-mini come with ATmega328P and it has a **Brown-Out Detector (BOD)**, which monitors the supply voltage at the time of **Sleep Mode**.

Power Management and Sleep Modes:

Sleep modes enable the application to shut down unused modules in the MCU, thereby saving power. The AVR provides various sleep modes allowing the user to tailor the power consumption to the application's requirements. There are **six sleep modes** in ATmega328P:

Table 7-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

	Active Clock Domains					Oscillators		Wake-up Sources							
Sleep Mode	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other/O	Software BOD Disable
Idle			X	X	X	X	X(2)	X	X	X	X	X	X	X	
ADC Noise Reduction				X	X	X	X(2)	X(3)	X	X(2)	X	X	X		
Power-down								X(3)	X				X		X
Power-save					X		X(2)	X(3)	X	X			X		X
Standby(1)						X		X(3)	X				X		X
Extended Standby					X(2)	X	X(2)	X(3)	X	X			X		X

Notes: 1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT1 and INT0, only level interrupt.

Power Management and Sleep Modes:

Table 7-1. Active Clock Domains and Wake-up Sources in the Different Sleep Modes.

	Active Clock Domains					Oscillators		Wake-up Sources							
Sleep Mode	clk_{CPU}	clk_{FLASH}	clk_{IO}	clk_{ADC}	clk_{ASY}	Main Clock Source Enabled	Timer Oscillator Enabled	INT1, INT0 and Pin Change	TWI Address Match	Timer2	SPM/EEPROM Ready	ADC	WDT	Other/O	Software BOD Disable
Idle			X	X	X	X	X(2)	X	X	X	X	X	X	X	
ADC Noise Reduction				X	X	X	X(2)	X(3)	X	X(2)	X	X	X		
Power-down								X(3)	X				X		X
Power-save					X		X(2)	X(3)	X	X			X		X
Standby(1)						X		X(3)	X				X		X
Extended Standby					X(2)	X	X(2)	X(3)	X	X			X		X

Notes: 1. Only recommended with external crystal or resonator selected as clock source.
2. If Timer/Counter2 is running in asynchronous mode.
3. For INT1 and INT0, only level interrupt.

Register Description:

SMCR – Sleep Mode Control Register

The Sleep Mode Control Register (SMCR) contains control bits for power management. The function of these bits (SM2:0) is shown in the next slide.

For entering into any of the sleep modes, we need to enable the sleep bit in the Sleep Mode Control Register (SMCR.SE). Then, the Sleep Mode Select bits select any 1 of the sleep modes among 6 different sleep modes, viz. Idle, ADC Noise Reduction, Power-Down, Power-Save, Standby, and External Standby.

An internal or external Arduino interrupt or a Reset can wake up the Arduino from sleep mode.

Bit	7	6	5	4	3	2	1	0	
0x33 (0x53)	–	–	–	–	SM2	SM1	SM0	SE	SMCR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Register Description:

Table 7-2. Sleep Mode Select

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby(1)
1	1	1	External Standby(1)

Note: 1. Standby mode is only recommended for use with external crystals or resonators.

Idle Mode:

For entering into the Idle sleep mode, write the SM[2,0] bits of the controller ‘000’. This mode stops the CPU but allows the SPI, 2-wire serial interface, USART, Watchdog, counters, and analog comparator to operate.

Idle mode basically stops the CLKCPU and CLKFLASH.

Arduino can be waked up at any time by using external or internal interrupts.

Arduino Code for Idle Sleep Mode:

```
LowPower.idle(SLEEP_8S, ADC_OFF, TIMER2_OFF, TIMER1_OFF,
TIMER0_OFF, SPI_OFF, USART0_OFF, TWI_OFF);
```

DC Noise Reduction Mode:

To use this sleep mode, write the SM[2,0] bit to ‘001’. The mode stops the CPU but allows the ADC, external interrupt, USART, 2-wire serial interface, Watchdog, and counters to operate. ADC Noise Reduction mode basically stops the CLKCPU, CLKI/O, and CLKFLASH. We can wake up the controller from the ADC Noise Reduction mode by the following methods:

- External Reset
- Watchdog System Reset
- Watchdog Interrupt
- Brown-out Reset
- 2-wire Serial Interface address match
- External level interrupt on INT
- Pin change interrupt
- Timer/Counter interrupt
- SPM/EEPROM ready interrupt

Power-Down Mode:

Power-Down Mode stops all the generated clocks and allows only the operation of asynchronous modules. It can be enabled by writing the SM[2,0] bits to ‘010’. In this mode, the external oscillator turns OFF, but the 2-wire serial interface, watchdog, and external interrupt continue to operate. It can be disabled by only one of the methods below:

- External Reset
- Watchdog System Reset
- Watchdog Interrupt
- Brown-out Reset
- 2-wire Serial Interface address match
- External level interrupt on INT
- Pin change interrupt

Standard Low Power Example:

This is the simplest way of implementing the Low Power mode. It will use the LED as an indicator to tell if the device is in an active state or sleep state. The device will be in a sleep state for 5 seconds.

```
#include "ArduinoLowPower.h"

void setup() {
  pinMode(LED_BUILTIN, OUTPUT); }

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
  LowPower.sleep(5000);
}
```

Here we can also change a line of the code to perform the Deep Sleep mode of the device.

```
//LowPower.sleep(5000);
LowPower.deepSleep(5000);
```

Low Power-Power Down Mode Example:

In this sketch, the Arduino blinks an LED for 2 s and is then powered down for 2 s, and during that time the ADC and Brown-Out Detect (BOD) are disabled. When powered down, the Arduino's current drops from 14 mA, down to just 6 µA!

```
#include "LowPower.h"

void setup() {
  pinMode(13,OUTPUT);
}

void loop() {
  digitalWrite(13,HIGH);
  delay(2000);
  digitalWrite(13,LOW);
  LowPower.powerDown(SLEEP_2S, ADC_OFF, BOD_OFF);
}
```

Let’s load this sketch onto our Arduino, which is running with a 5 V supply at 16 MHz. To see how little current is needed in sleep mode, we are using an Arduino ATmega328P board to minimize the current.

Arduino Code for Power-Down Periodic Mode:

LowPower.powerDown(SLEEP_8S, ADC_OFF, BOD_OFF);

The code is used to turn on the power-down mode. By using the above code, the Arduino will go to sleep for eight seconds and wake up automatically.

We can also use the power-down mode with an interrupt, where the Arduino will go to sleep but only wakes up when an external or internal interrupt is provided.

Arduino Code for Power-Down Interrupt Mode:

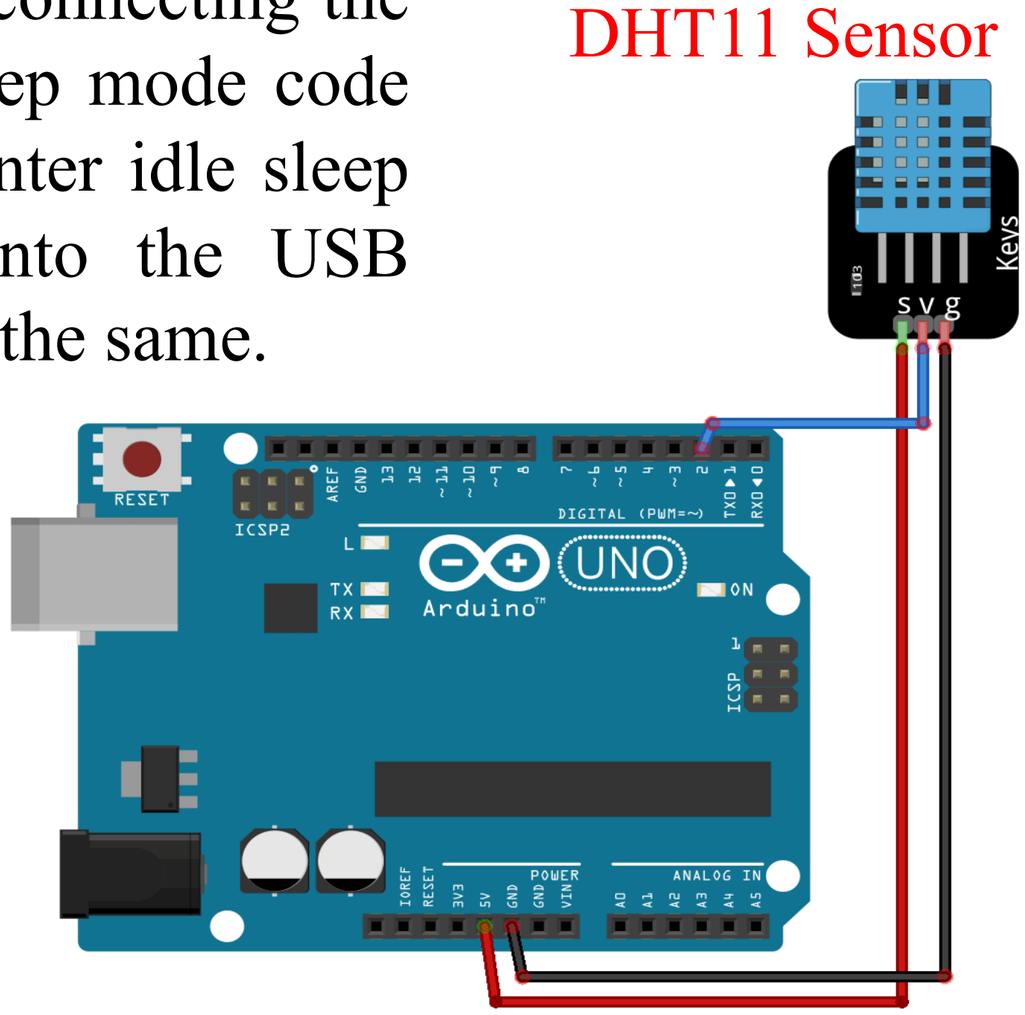
```
void loop() {
  // Allow the wake-up pin to trigger an interrupt on low.
  attachInterrupt(0, wakeUp, LOW);
  LowPower.powerDown(SLEEP_FOREVER, ADC_OFF, BOD_OFF);
  // Disable external pin interrupt on the wake-up pin.
  detachInterrupt(0);
  // Do something here
}
```

Power-Save Mode: To enter the power-save mode, we need to write the SM[2,0] pin to ‘011’. This sleep mode is like the power-down mode, only with one exception i.e., if the timer/counter is enabled, it will remain in a running state even at the time of sleep. The device can be waked up by using the timer overflow bit, TOVn.
If you are not using the time/counter, it is recommended to use Power-down mode instead of power-save mode.
Standby Mode: The standby mode is identical to the Power-Down mode, the only difference in between them is the external oscillator kept running in this mode. For enabling this mode, write the SM[2,0] pin to ‘110’.
Extended Standby Mode: This mode is like the power-save mode only with one exception that the oscillator is kept running. The device will enter the Extended Standby mode when we write the SM[2,0] pin to ‘111’. The device will take six clock cycles to wake up from the extended standby mode.

Practical Application with a DHT11 Sensor:

Below are the requirements for this project, after connecting the circuit as per the circuit diagram. Upload the sleep mode code into Arduino using Arduino IDE. Arduino will enter idle sleep mode. Then check the current consumption into the USB ammeter. Else, you can also use a clamp meter for the same.

In this setup, to demonstrate Arduino Deep sleep modes, the Arduino is plugged into the **USB ammeter**. Then the USB ammeter is plugged into the USB port of the laptop.
The data pin of the DHT11 sensor is attached to the D2 pin of the Arduino.



Code Explanation

The code starts by including the library for the DHT11 sensor and the Low Power library. For downloading the Low Power library follow the link (*https://github.com/rocketscream/Low-Power*).

Then we defined the Arduino pin number to which the data pin of the DHT11 is connected and created a DHT object.

```
#include <dht.h>
#include <LowPower.h>

#define dataPin 2
dht DHT;
```

USB Ammeter

Code Explanation

In the void setup function, we have initiated the serial communication by using serial.begin(9600), here the 9600 is the baud rate. We are using Arduino's built-in LED as an indicator for the sleep mode. So, we have set the pin as output, and digital write low.

```
void setup() {
  Serial.begin(9600);
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(LED_BUILTIN, LOW);
}
```

Code Explanation

In the void loop function, we are making the built-in LED HIGH and the reading the temperature and humidity data from the sensor. Here, DHT.read11(); command is reading the data from the sensor. Once data is calculated, we can check the values by saving them into any variable. Here, we have taken two float type variables ‘t’ and ‘h’. Hence, the temperature and humidity data is printed serially on the serial monitor.

```
void loop() {
  Serial.println("Get Data From DHT11");
  delay(1000);
  digitalWrite(LED_BUILTIN,HIGH);
  int readData = DHT.read11(dataPin); // DHT11
```

Code Explanation

```
float t = DHT.temperature;
float h = DHT.humidity;

Serial.print("Temperature = ");
Serial.print(t);
Serial.print(" C | ");

Serial.print("Humidity = ");
Serial.print(h);
Serial.println(" % ");

delay(2000);
```

Code Explanation

Before enabling the sleep mode, we are printing "Arduino: - I am going for a Nap" and making the built-in LED Low. After that Arduino sleep mode is enabled by using the command mentioned below in the code.
The below code enables the idle periodic sleep mode of the Arduino and gives a sleep of eight seconds. It turns the ADC, Timers, SPI, USART, and 2-wire interface into the OFF condition.

Then it automatically wakes up Arduino from sleep after 8 seconds and prints “Arduino:- Hey I just Woke up”.

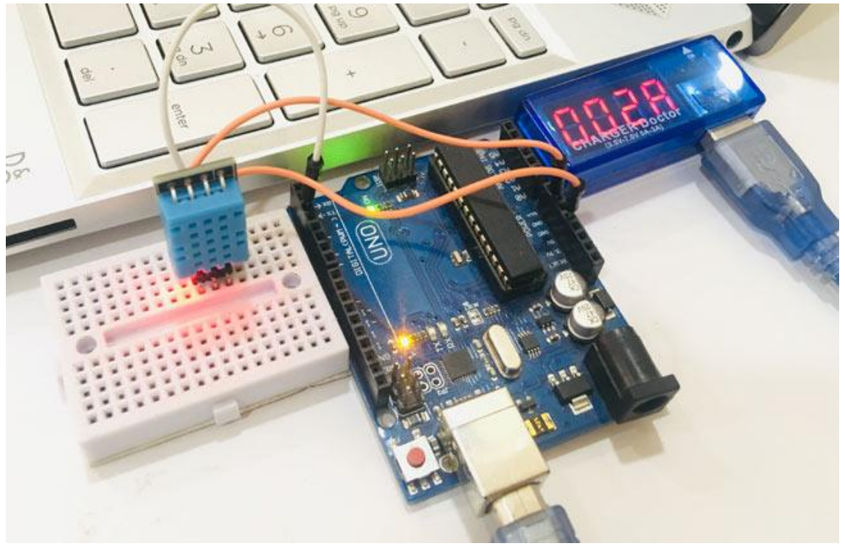
Code Explanation

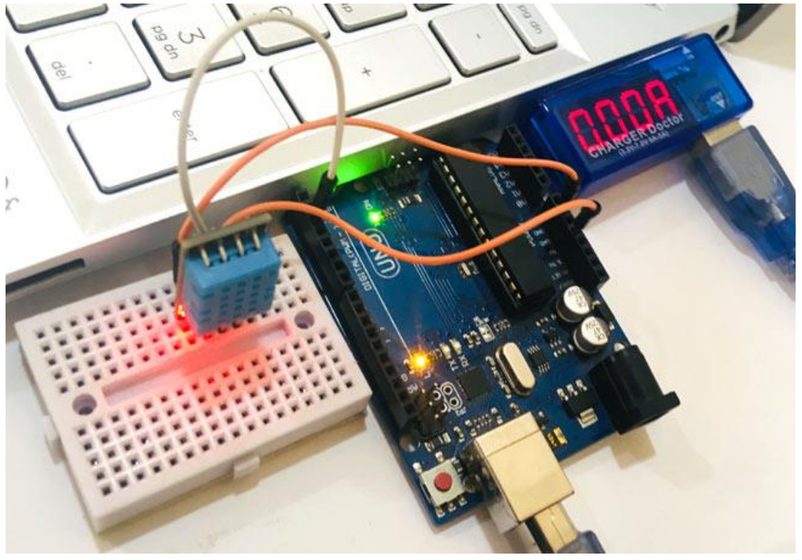
```
Serial.println("Arduino:- I am going for a Nap");
delay(1000);
digitalWrite(LED_BUILTIN,LOW);
LowPower.idle(SLEEP_8S, ADC_OFF, TIMER2_OFF, TIMER1_OFF, TIMER0_OFF,
SPI_OFF, USART0_OFF, TWI_OFF);
Serial.println("Arduino:- Hey I just Woke up");
Serial.println("");
delay(2000);
}
```

So, by using this code Arduino will only wake up for 28 seconds in a minute and will remain in sleep mode for the rest of the 32 seconds, which significantly reduces the power consumed by the Arduino weather station. Therefore, if we use the Arduino with the sleep mode, we can approximately double the device runtime.

Experimental Setup with USB Ammeter

USB ammeter is a plug-and-play device used to measure the voltage and current from any USB port. The dongle plugs in between the USB power supply (computer USB port) and the USB device (Arduino). This device has a 0.05 Ω resistor in-line with the power pin through which it measures the value of the current drawn. The device comes with four seven-segment displays, which instantly display the values of current and voltage consumed by the attached device. These values flip in an interval of every 3 seconds.





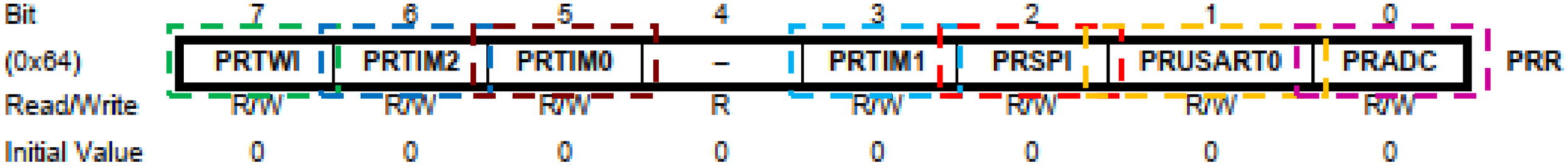
Register Description:

Table 7-2. Sleep Mode Select

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby(1)
1	1	1	External Standby(1)

Note: 1. Standby mode is only recommended for use with external crystals or resonators.

PRR: Power Reduction Register:



Bit	7	6	5	4	3	2	1	0	
(0x64)	PRTWI	PRTIM2	PRTIM0	–	PRTIM1	PRSPI	PRUSART0	PRADC	PRR
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Bit 7 - PRTWI: Power Reduction TWI
- Bit 6 - PRTIM2: Power Reduction Timer/Counter2
- Bit 5 - PRTIM0: Power Reduction Timer/Counter0
- Bit 4 - Res: Reserved bit
- Bit 3 - PRTIM1: Power Reduction Timer/Counter1
- Bit 2 - PRSPI: Power Reduction Serial Peripheral Interface
- Bit 1 - PRUSART0: Power Reduction USART0
- Bit 0 - PRADC: Power Reduction ADC

Setting up PRR using Assembly Language:

0b10101100 =0xAC

```
LDI R10, 0xAC;
OUT PRR, R10;
IN R10, PRR;
```

Watchdog Timer (WDT)

The Watchdog Timer (WDT) in the ATmega328P microcontroller is a system timer that can be used to reset the microcontroller if it gets stuck due to a software error or hangs due to unexpected conditions. It is an important feature for ensuring the reliability of embedded systems.

```
#include <avr/io.h>
#include <avr/wdt.h>
void setup() {
// Configure the WDT to reset after 1 second
wdt_enable(WDTO_1S);
}
void loop() {
// Main program logic
// Reset the WDT periodically to prevent a system reset
wdt_reset();
// Other code... }
```

In this example, the WDT is configured to reset the microcontroller if it is not reset within 1 second, ensuring the system does not remain stuck in an infinite loop or error state.

QUESTION: Prepare an assembly program to set the Sleep Mode Control Register (SMCR) so that the system remains in the power-save mode and power reduction mode (PRR) for its Timer0, Timer1, and SPI. Use the following table for the SMx bits of the SMCR register. The PRR register bits are shown, too.

Bit	7	6	5	4	3	2	1	0	
0x33 (0x53)	–	–	–	–	SM2	SM1	SM0	SE	SMCR
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
(0x64)	PRTWI	PRTIM2	PRTIM0	–	PRTIM1	PRSPI	PRUSART0	PRADC	PRR
Read/Write	R/W	R/W	R/W	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power-down
0	1	1	Power-save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby(1)
1	1	1	External Standby(1)

Example 1

Compute the total time for which the device is in low-power mode as per the following program if the loop continues for 20 cycles.

```
#include "ArduinoLowPower.h"

void setup() {
      pinMode(LED_BUILTIN, OUTPUT); }

void loop() {
      digitalWrite(LED_BUILTIN, HIGH);
      delay(1000);
      digitalWrite(LED_BUILTIN, LOW);
      delay(1000);
      LowPower.deepSleep(8000);  }
```

Answer:

If the above program is run then the time for which the device is in low-power deep sleep mode is 8000 ms or 8 s. Hence, if the loop continues for 20 cycles, then the device is in low-power deep sleep mode for 8×20 = 160 s.

Example 2

Compute the duration that the Arduino blinks an LED connected to pin 10 and makes the power down the system. When powered down, the Arduino's current drops from 10 mA to just 8 μA. If the Arduino's supply voltage is 3.3 V, then **compute** the amount of power that is saved by the Arduino during this power-down mode.

```
#include "LowPower.h"

void setup() {
  pinMode(10, OUTPUT); }

void loop() {
  digitalWrite(10, HIGH);
  delay(1000);
  digitalWrite(10, LOW);
  LowPower.powerDown(SLEEP_4S, ADC_OFF, BOD_OFF, SPI_OFF, TWI_OFF);
}
```

Solution:

If the above program is run, then the time for which the LED is ON is 1000 ms or 1 s because the digitalWrite() function sends a 'HIGH' signal to pin 10, where an LED is connected, and then the delay() function makes a delay of 1000 ms. After that, the digitalWrite() function sends a 'LOW' signal to pin 10. After that, there is no delay() function, but the device goes into power-down mode for 4 s. As such, the duration that the Arduino blinks an LED connected to pin 10 is every 5 s, that is, the LED is ON for 1 s and OFF for 4 s, and this loop continues.

Initially, when the power is not down then the device draws a 10 mA current. Hence, it consumes a power of 3.3×10 = 33 mW.
When the device is in low-power power-down mode, then the device draws an 8 μA current. Hence, it consumes a power of 3.3×8 = 26.4 μW = 0.0264 mW.
So, the amount of power that is saved by the Arduino during the low-power power-down mode = 33 – 0.0264 = 32.9736 mW.

Example 3

Determine the output of the following program. **Compute** power and energy consumption during the ON and OFF conditions as well as SLEEP modes in idle and power down states. **Compute** power savings.

```
#include “LowPower.h”

void loop() {
        pinMode(3, OUTPUT);  }

void setup() {
        digitalWrite(3, HIGH);
        delay(4000);   // USB Meter readings = 4.971 V, 0.758 A)
        digitalWrite(3, LOW);
        delay(2000); // USB Meter readings = 5.017 V, 0.125 A)
        LowPower.idle(SLEEP_8S, ADC_OFF, SPI_OFF); // USB Meter readings = 5.15 V, 0.001 A)
        LowPower.powerDown(SLEEP2S, ADC_OFF, BOD_OFF); // USB Meter readings = 5.08 V, 0.005 A)   }
```

Solution:

If the above program is run then the time for which the LED is ON is 4000 ms or 4 s because the digitalWrite() function sends a ‘HIGH’ signal to pin 10 where an LED is connected and then the delay() function makes a delay of 4000 ms. After that, the digitalWrite() function sends a ‘LOW’ signal to pin 10. After that, there is a delay() function, so the LED is turned OFF for 2000 ms or 2 s. Then the device goes into idle mode for 8 s. Finally, the device goes into power-down mode for 2 s. As such, the duration that the Arduino blinks an LED connected to pin 10 is every 16 s, that is, the LED is ON for 4 s and OFF for 2 + 8 + 2 = 12 s, and this loop continues.

Example 3

According to the comments in the code, the USB meter readings are provided.
Initially, when the device is in the HIGH state and the LED is ON, it draws a 0.758 A current at a voltage of 4.971 V for a duration of 4 s. Hence, it consumes a power of $P_1 = 4.971 \times 0.758 = 3.768$ W and an energy of $E_1 = 3.768 \times 4 = 15.072$ J.
When the device is in the LOW state and the LED is OFF, it draws a 0.125 A of current at a voltage of 5.017 V for a duration of 2 s. Hence, it consumes a power of $P_2 = 5.017 \times 0.125 = 0.627$ W and an energy of $E_2 = 0.627 \times 2 = 1.254$ J.
When the device is in the idle mode and the LED is OFF, it draws a 0.001 A of current at a voltage of 5.15 V for a duration of 8 s. Hence, it consumes a power of $P_3 = 5.15 \times 0.001$ A $= 0.00515$ W and an energy of $E_3 = 0.00515 \times 8 = 0.0412$ J.
When the device is in the low-power power-down mode, it draws a 0.005 A of current at a voltage of 5.07 V for a duration of 2 s. Hence, it consumes a power of $P_4 = 5.08 \times 0.005$ A $= 0.0254$ W and an energy of $E_4 = 0.0254 \times 2 = 0.0508$ J.
So, the amount of energy that is saved by the Arduino during the idle and low-power power-down mode from the LOW state are $\frac{1.254-0.0412}{1.254} \times 100\% = 96.72\%$ and $\frac{1.254-0.0508}{1.254} \times 100\% = 95.95\%$, respectively.

Example 4

Compute power and energy consumption during the ON, OFF, and SLEEP modes, according to the following USB meter readings. **Compute** the percentage of power and energy savings for the OFF and SLEEP conditions if the device stays in all modes are in for 4 s.

LED ON Mode

LED SLEEP Mode

LED OFF Mode

Solution:

According to the USB meter readings:

When the LED is in the ON state, it draws a 0.117 A current at a voltage of 5.002 V for a duration of 4 s. Hence, it consumes a power of $P_1 = V_1 I_1 = 5.002 \times 0.117 = 0.58523$ W and an energy of $E_1 = P_1 t_1 = 0.\ 58523 \times 4 = 2.341$ J.

Example 4

When the LED is in SLEEP mode, it draws a 0.008 A current at a voltage of 5.069 V for a duration of 4 s. Thus, it consumes a power of $P_2 = V_2I_2 = 5.069 \times 0.008 = 0.04055$ W and an energy of $E_2 = P_2t_2 = 0.04055 \times 4 = 0.1622$ J.

When the LED is in the OFF state, it draws a 0.019 A current at a voltage of 5.063 V for a duration of 4 s. So, it consumes a power of $P_3 = V_3I_3 = 5.063 \times 0.019 = 0.0962$ W and an energy of $E_3 = P_3t_3 = 0.0962 \times 4 = 0.3848$ J.

So, the amount of power that is saved by the Arduino during the SLEEP mode from the LOW state is $\frac{0.0962 - 0.04055}{0.0962} \times 100\% = 57.85\%$.

And, the amount of energy that is saved by the Arduino during the SLEEP mode from the LOW state is $\frac{0.3848 - 0.1622}{0.3848} \times 100\% = 57.85\%$.

Thanks for Attending….

[1] Reducing Arduino's Power Consumption Part 1 – YouTube
https://www.youtube.com/watch?v=ktvUunBQQD0